

Step A and the frontend plan

Part 1 closes Phase 1 of the Solana work: switch the live app to devnet. Part 2 shows where that sits in the whole backend plan. Part 3 explains how it works inside. Part 4 is the step-by-step. Part 5 plans the frontend: split the single page into routes and make it work well on phones.

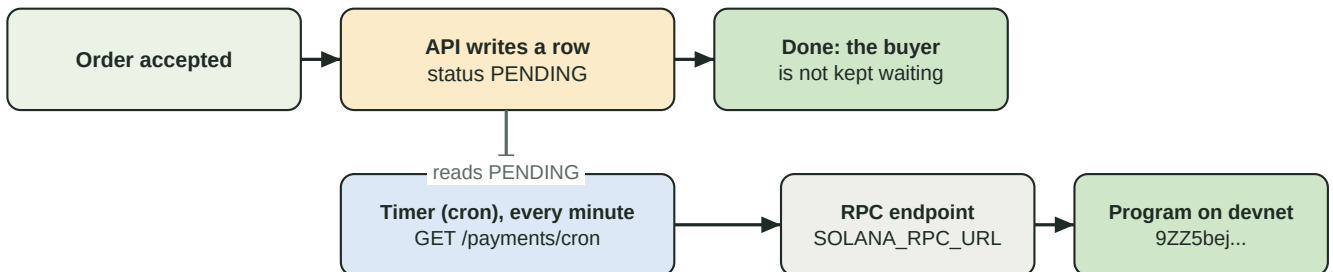
In plain words

- **The code for Solana is finished and tested.** What is left in Phase 1 is five settings on Vercel and a timer. No programming.
- **Nothing here uses real money.** Devnet is Solana's practice network; its coins are free.
- **The frontend does not have to wait.** Splitting it into routes can start now. Only its 'on-chain proof' badge needs Step A first.

BEFORE (today): the mock provider answers instantly, nothing leaves our server



AFTER (Step A): the row waits, a timer sends it to Solana devnet



When it lands the row turns CONFIRMED and stores the transaction signature you can open in the explorer.

Figure 1. The buyer never waits for the blockchain. The API only writes a note; a timer delivers it.

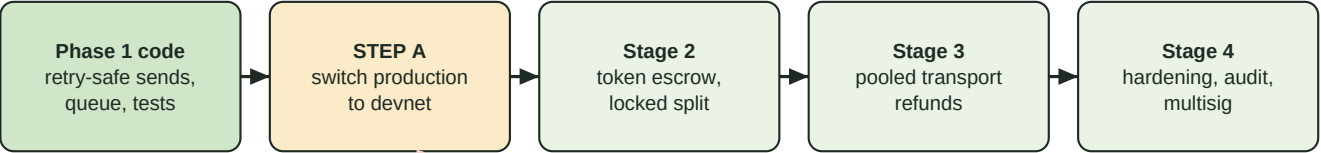
What stays the same

- Orders, listings, matching and every page of the app behave as before.
- If Solana is slow or down, orders still go through. The failed note is kept and retried.
- Orders accepted before the switch have no commit note, so nothing is sent for them. That is intended.

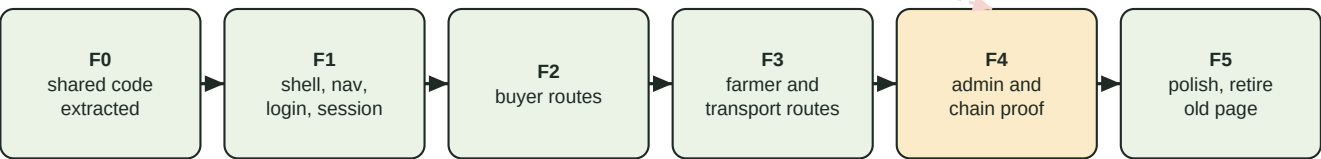
Part 2: where Step A fits

The marketplace is built in stages. Each stage changes what the blockchain is trusted with. Step A is the bridge between 'it works on my laptop' and 'the live app really writes to a public ledger'. Every later stage assumes it is done, because it proves the real path (app, queue, key, network, program) end to end before anything valuable depends on it.

BACKEND AND SOLANA



FRONTEND



F0 to F3 do not wait for anything on the backend: they can start today, in parallel with Step A.

Stage 2 adds payment screens later; the F1 shell is where they will plug in.

Figure 2. Backend stages on top, frontend phases below. The dashed arrow is the only hard link between them.

Step	What it does	Status	Needs
Phase 1 code	Retry-safe sends, queue, cron route, 6 program tests, devnet deploy	Done and pushed	Nothing
Step A	Make the live backend actually use devnet	This guide	5 settings, redeploy, a timer
Stage 2	Token escrow: the program holds the buyer's payment and releases it by a split locked at order time (farmer, transporter, platform fee, capped)	Planned	Step A; Anchor 0.30.1 to 1.2; your decisions on fee, who sets transport price, silent-buyer rule, disputes
Stage 3	Pooled transport as a shared pot with a target, a deadline and refunds	Planned	Stage 2
Stage 4	Hardening: paid RPC, multisig upgrade authority, external audit, mainnet decision	Planned	Stages 2 and 3, real usage

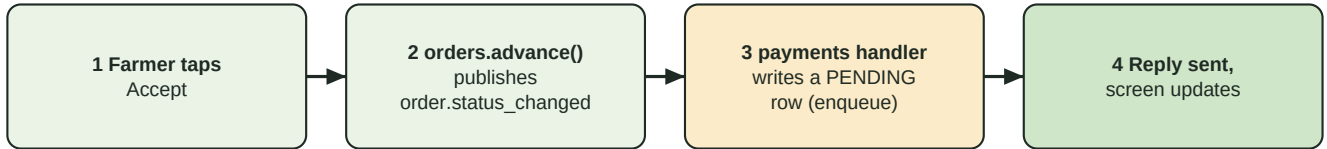
Why not go straight to Stage 2?

Escrow adds money, token accounts and a fee. If the plain path (queue, key, network) has a problem, you would be debugging it and the new money logic together. Step A is a one-hour check that removes the first set of unknowns.

Part 3: how it works inside

Two requests are involved, and keeping them apart is the whole idea. The first is the user's own tap and must be fast. The second is a timer that does the slow work.

INSIDE THE REQUEST: fast, the buyer never waits for Solana



LATER, ON A TIMER: a separate request, up to a minute afterwards

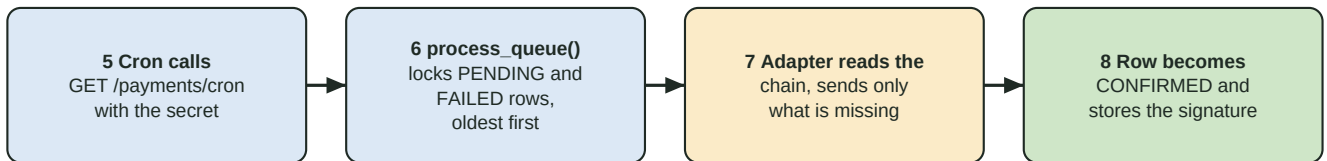


Figure 3. The dashed line separates what the user waits for from what happens later.

Step	Where in the code	Why it is built this way
2 to 3	orders/service.py advance(); payments/service.py on_status_changed and enqueue	The payment note is written in the same database transaction as the order change, so there is never an order without its note, or the reverse.
3	enqueue(): mock provider runs inline, a real chain leaves the row PENDING	The mock is instant, so it needs no timer. A real network call can take seconds, so it must stay out of the user's request.
5	payments/routes.py run_queue: answers only if the Authorization header equals Bearer CRON_SECRET	The route is public on the internet, so it is locked by a password. If CRON_SECRET is not set it is switched off completely.
6	process_queue(): rows locked with skip_locked, sorted commit, then attestations by position, then settle	Two timers overlapping must not both send. Rows made in one request share a timestamp, so the order is stated, not left to chance.
7	solana_adapter.py: reads the order's account first, sends only the missing step	A reply can be lost after a transaction already landed. Reading first, plus the program's expected-position check, makes a repeat harmless.
8	row status CONFIRMED, external_id = the signature	The signature is the receipt: anyone can open it in the explorer.

What if a step fails?

The row becomes FAILED with the reason stored (for example 'rate limited'). The order itself is never blocked or rolled back. The next timer tick retries every PENDING and FAILED row, oldest first, so a failed commit is repeated before the steps that depend on it.

Part 4: do Step A

4a. What is already done

Item	Status	Detail
Backend signing key	Created	Public address 8dxtAKX8UUDCeUkbttr6foBXedV7ZU5X23iUXR9TAacA. File: ~/.config/solana/farmers-market-backend-devnet.json (permissions 600, outside the git repo). Devnet only. It is separate from your laptop wallet so a leak of one does not expose the other.
Funding	1 SOL on devnet	Sent from your laptop wallet. Each order needs about 0.00125 SOL of rent plus 0.000005 SOL per step, so 1 SOL covers hundreds of test orders.
Program	Deployed	9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU. 13 of 13 smoke checks pass on devnet.
Queue and timer route	In the live code	Pushed and deployed. The route stays locked (401) until CRON_SECRET exists.
The five settings on Vercel	NOT set	Blocked by the safety rules of my tool, because it means writing secrets to your Vercel account. You do this part.

4b. The five settings

The five settings and what each one does

PAYMENT_PROVIDER = solana	the switch: use the chain instead of the mock
SOLANA_PROGRAM_ID	which program to talk to
SOLANA_RPC_URL	which door to the network (public, or a keyed one)
SOLANA_AUTHORITY_KEYPAIR_JSON	the backend's signing key. SECRET.
CRON_SECRET	password the timer must send. SECRET.

Red = a leaked value lets someone act as your backend. Mark both Sensitive in Vercel.

Figure 4. Three plain settings, two secrets.

Name	Value	Read by	If it is wrong
PAYMENT_PROVIDER	solana	payments/service.py get_provider	Stays on the mock: nothing reaches the chain, no error
SOLANA_PROGRAM_ID	9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU	solana_adapter.py from_env	Rows FAILED: program not found
SOLANA_RPC_URL	https://api.devnet.solana.com (works, rate-limited; a keyed Helius URL is steadier)	solana_adapter.py	Rows FAILED with 429 or connection errors; retried next tick
SOLANA_AUTHORITY_KEYPAIR_JSON	the whole contents of the key file: 64 numbers in square brackets. Sensitive	solana_adapter.py from_env	Backend fails to start sending: rows FAILED with a key error
CRON_SECRET	a long random string you make up. Sensitive	payments/routes.py run_queue	Timer gets 401; rows stay PENDING

4c. Choose one way to add them

Way 1: Vercel dashboard (easiest to see). Open vercel.com, project **farmers-market-place** (the backend, not the -le5n one). Settings, then Environment Variables. For each row: type the name and value, tick **Production**, and for the two secrets tick **Sensitive**. Save.

```
$ cat ~/.config/solana/farmers-market-backend-devnet.json
[12,200,...64 numbers...,31]
# make a cron secret: run this, copy the output
$ python3 -c "import secrets;print(secrets.token_urlsafe(32))"
```

Way 2: one command per setting. Type each line into the Claude Code prompt starting with the exclamation mark, so it runs in this session with your approval. The value goes in through a pipe, so it is not typed into the command itself.

```
$ cd ~/Dev/singapore/api
$ printf '%s' solana | npx vercel env add PAYMENT_PROVIDER production --project farmers-market-place
$ printf '%s' 9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU | npx vercel env add SOLANA_PROGRAM_ID production --
$ printf '%s' https://api.devnet.solana.com | npx vercel env add SOLANA_RPC_URL production --project farmers-ma
$ cat ~/.config/solana/farmers-market-backend-devnet.json | npx vercel env add SOLANA_AUTHORITY_KEYPAIR_JSON pr
$ printf '%s' YOUR_RANDOM_STRING | npx vercel env add CRON_SECRET production --project farmers-market-place --s
```

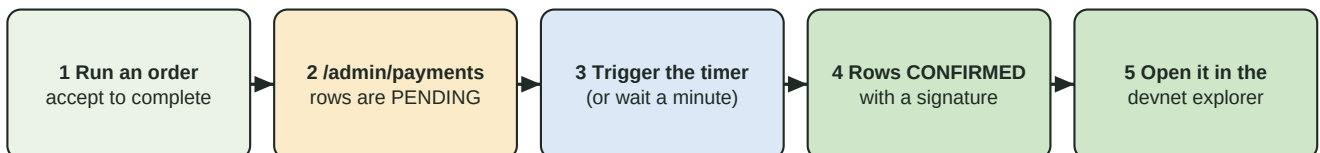
Check that the names landed (values stay hidden), then redeploy:

```
$ npx vercel env ls production --project farmers-market-place | grep -E 'SOLANA|CRON|PAYMENT'
```

Settings only apply to a new deployment. In the dashboard open the backend project, Deployments, the latest one, the three-dot menu, **Redeploy**. Wait for Ready.

4d. Prove it works

How you know it worked: follow one order from the app to the explorer



Success = every row of a finished order is CONFIRMED: one commit, one per attestation, one settle.

A row that says FAILED is not a disaster: the reason is stored and the next tick retries it.

Figure 5. One order, followed all the way to the explorer.

Run these after the redeploy is Ready. Replace the placeholders.

```
# get an admin token (this reseeds the demo data: fine while there are no real users)
$ curl -s -X POST 'https://farmers-market-place-nine.vercel.app/demo/seed?role=admin'
# in the web app: log in as buyer, create an order; farmer accepts; run it to COMPLETED
# then look at the notes:
$ curl -s -H 'Authorization: Bearer ADMIN_TOKEN' https://farmers-market-place-nine.vercel.app/admin/payments
[{"purpose":"order","status":"PENDING","provider":"solana",...}]
# trigger the timer by hand instead of waiting
$ curl -s -H 'Authorization: Bearer YOUR_CRON_SECRET' https://farmers-market-place-nine.vercel.app/payments/cron
{"processed":5}
```

Run the admin call again: the rows say CONFIRMED and each has an external_id. That is the transaction signature. Open it:

```
$ https://explorer.solana.com/tx/<external_id>?cluster=devnet
```

You should see Result: Success and in the logs Instruction: CommitOrder, RecordAttestation or Settle. The signer is the backend key address from 4a.

If something looks wrong

What you see	Meaning	Fix
Timer call returns 401	CRON_SECRET missing, wrong, or you did not redeploy	Check the name, re-add, redeploy, resend with the exact value
Rows stay PENDING	Nothing has called the timer yet	Call /payments/cron by hand, or schedule it (below)
FAILED, 429	The public endpoint is rate-limiting	Call the timer again, or use a keyed RPC URL
FAILED, insufficient funds	Backend key is out of devnet SOL	solana -u devnet transfer 8dxtAKX8UUDCeUkb ttr6foBXedV7ZU5X23iUXR9TAacA 1
FAILED, AccountNotFound on attest or settle	That order's commit has not landed yet	Nothing: the next tick replays commit first, then the rest, in order
Order screens show a 500	Database is missing migration 0004	alembic upgrade head with the unpooled Neon URL

Run the timer automatically

A recurring call to /payments/cron with the header Authorization: Bearer YOUR_CRON_SECRET.

- **Vercel Cron:** a once-a-minute schedule needs a Pro plan; Hobby only allows daily and would reject the whole deploy otherwise. Tell me your plan and I will add it.
- **Any outside scheduler** that can send that header every minute, for example a GitHub Actions schedule.
- Until then, call it by hand when you test.

Undo, in two minutes

Change PAYMENT_PROVIDER back to mock (or delete it) and redeploy. New orders go back to instant mock rows. Stored notes stay in the database and do no harm.

Part 5: frontend planning phase

Goal: replace the one-page demo with a proper multi-route app that works well on a phone, without breaking the live site on the way. This is a plan, not built yet.

TODAY: one file does everything, and forgets you on refresh

app/page.tsx (209 lines)
one URL: /
every role's screen sits inside it,
shown or hidden by session.role

Session in React state only
refresh = signed out

One 91-line stylesheet,
two breakpoints (620, 768)

swr installed, never used

No deep links: a lot, an order
or a dispute has no address

It works and is easy to demo. It cannot be bookmarked, shared by link, or opened from a phone notification.

Tomorrow's Stage 2 payment screens would make that single file much larger. Splitting now is cheaper than later.

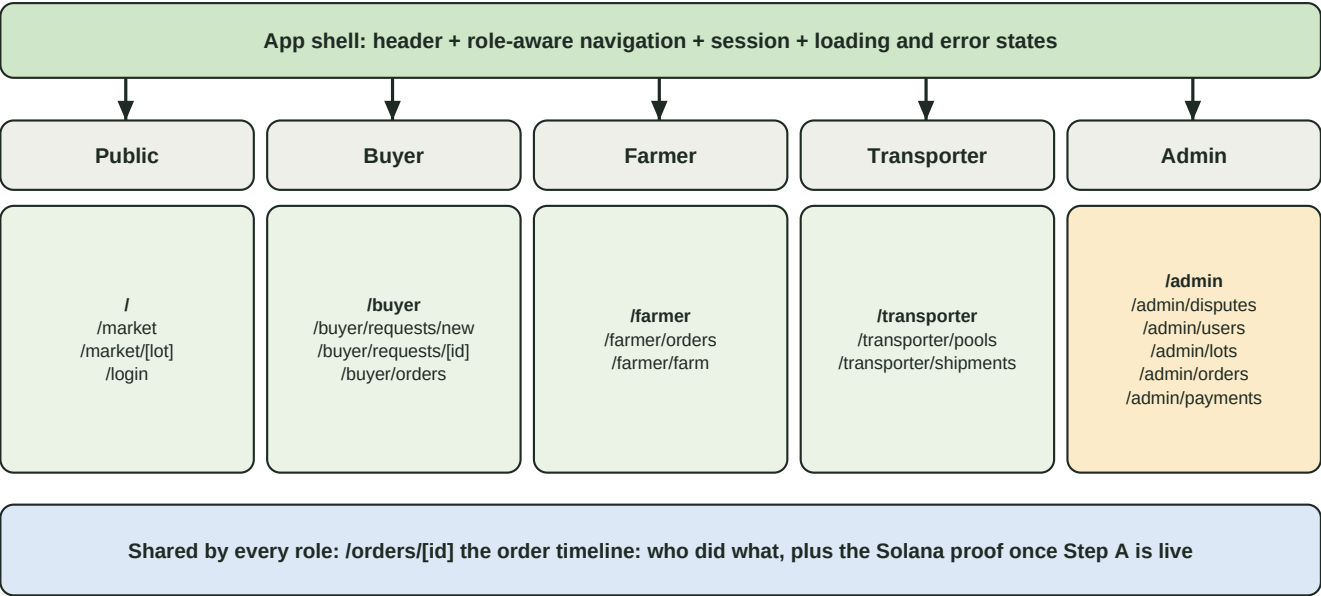
Figure 6. What the frontend is today (read from apps/web).

What is wrong with one page

Problem	Why it matters
No addresses	An order or a lot cannot be linked to, bookmarked or opened from a notification. Support questions become 'scroll down to My orders'.
Session lost on refresh	On a phone, switching apps often reloads the page, which signs the user out. This is the biggest phone problem.
All roles in one file	Every change risks another role's screen. Stage 2 would add checkout and fee screens to the same file.
One long scroll on a small screen	Role cards stack into a very tall page; there is no navigation to jump between 'Orders' and 'Market'.
Data loading in many functions	load* helpers called by hand after each action; easy to forget one and show stale data. The unused swr package is built for this.

The route map

PROPOSED ROUTES: one folder per role, one shared shell around all of them



Square brackets are the changing part: /orders/ord_123 opens that order. Admin payments is the only brand-new screen.

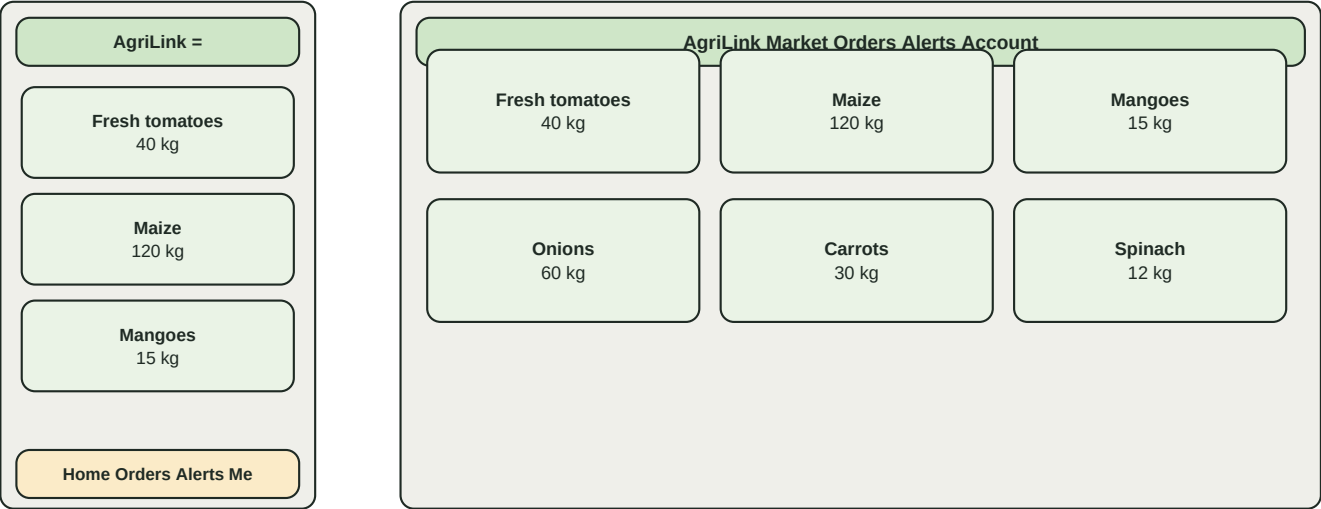
Figure 7. Every screen that exists today gets its own address. Only /admin/payments is new, and it uses the existing admin endpoints.

Where each existing section goes

Today (section in page.tsx)	Becomes
Available produce (everyone)	/market, with /market/[lot] for one lot and the order form
Open buyer request, Match plan	/buyer/requests/new, then /buyer/requests/[id] showing the match plan
Create direct order, My orders	Order form on /market/[lot]; list on /buyer/orders; one order on /orders/[id]
Farmer order queue, Farm location	/farmer/orders and /farmer/farm
Pooled transport, Shipment orders	/transporter/pools and /transporter/shipments
Disputes, User moderation, Listing moderation, All orders	/admin/disputes, /admin/users, /admin/lots, /admin/orders
Demo role buttons	/login (demo roles stay while the demo seed is on; real email login uses the existing /login and /register endpoints)

Made for phones

PHONE FIRST: the same pages, rearranged by screen width



Under 768 px

1 column, bottom tabs, big buttons

768 px and wider

top navigation, 2 to 3 columns, tables allowed

Figure 8. Same content, two arrangements. Phone first, then widen.

Rule	Detail
Mobile first CSS	Write the phone layout as the default; add wider layouts with min-width queries at 768 and 1100 px. Test at 360, 390, 768 and 1280 px.
Bottom tab bar on phones	4 or 5 role-specific tabs within thumb reach; the top bar only on wider screens. One component, two arrangements.
Touch targets	Buttons and links at least 44 px high; forms one column; inputs use the right keyboard (number, email) and 16 px text so iPhones do not zoom.
Lists as cards	Orders and lots are cards on phones. Tables only above 768 px (admin screens).
No horizontal scroll	Long IDs wrap or truncate with the full value on tap. Images use fixed aspect ratios so the page does not jump while loading.
Weak networks	Skeleton placeholders, plain error messages with a retry button, and no spinner that never ends. Server-fetched data cached per route.

How to build it, in order

The old page stays at / and keeps working until the very end. Each phase below can be pushed on its own; if one goes wrong the live app is untouched.

Phase	What	Done when	Needs backend?
F0 Prepare	Move api() to lib/api.ts, types to lib/types.ts, and add lib/session.tsx that saves the session in localStorage (wrapped in try/catch, as private mode can block it). Wire swr for data fetching or delete the dependency. No visible change.	npm run build passes; the old page behaves identically; refresh keeps you signed in	Maybe: a small GET /me so a stored token can be checked
F1 Shell	Root layout with header and role-aware navigation (top bar wide, bottom tabs narrow), /login, a guard that sends the wrong role to /login, shared loading and error components, mobile-first stylesheet.	Each role sees only its own nav; direct links to protected routes work after refresh	No
F2 Buyer	/market, /market/[lot], /buyer/requests/new and [id], /buyer/orders, /orders/[id] with the order timeline from the existing history endpoint.	A buyer can place and follow an order using only the new routes, on a phone	No
F3 Farmer and transporter	/farmer/orders, /farmer/farm, /transporter/pools, /transporter/shipments.	Accept, pickup, deliver and pool confirm all work from the new routes	No
F4 Admin and chain proof	/admin/disputes, users, lots, orders; /admin/payments (list, retry button); a 'Recorded on Solana' badge with explorer link on /orders/[id].	An order that went through Step A shows its signature link; admin sees FAILED rows and can retry	Yes: Step A live, plus a small GET /orders/{id}/payments visible to the order's participants
F5 Polish	Empty, loading and error states everywhere, keyboard and screen-reader pass, phone Lighthouse check, then replace the old page.tsx at / with the landing page.	No dead code left in page.tsx; Lighthouse accessibility and performance over 90 on a phone profile	No

Backend additions the frontend would ask for

Addition	Size	Why
GET /me	about 5 lines	Verify a stored token on app start and get the role; today the only way to learn the role is the demo seed reply
GET /orders/{id}/payments	about 15 lines	Lets a buyer, farmer or transporter see the chain status of their own order; today only admins can read payment rows
NEXT_PUBLIC_SOLANA_CLUSTER	one env var	Builds correct explorer links (devnet now, mainnet later) without hard-coding

Risks and open questions

Risk	Handling
Breaking the live demo	Old page stays at / until F5; ship one phase per push; check the Vercel preview URL of each phase before merging.
localStorage tokens	Acceptable for this demo-grade bearer token; for real users plan an httpOnly cookie. Noted, not solved here.
Demo seed in production	It wipes data and has no login. Turn ENABLE_DEMO_SEED off before real users and let /login use real accounts.
Scope creep into design	Keep the AgriLink look; this phase is structure and responsiveness, not a redesign.

Questions for you

1. Who is the first real user on a phone: farmers, buyers or transporters? That decides whether F2 or F3 goes first.
2. Should a farmer be able to add lots from the app? Today lots are created by the backend or seed only; it is a missing screen, not a layout task.
3. Is the legacy page `web/index.html` (served by the API) still needed? I would delete it in F5.
4. Languages: the new routes are a good moment to add one if a pilot country needs it.

The whole to-do, in order

#	Task	Who	Needs
1	Add the five Vercel settings (Part 4c) and redeploy	You	Vercel access
2	Run one order and check the explorer (Part 4d)	Me, once you say go	Step 1
3	Schedule the timer (Vercel Pro cron or an outside scheduler)	You tell me the plan; I add it	Step 1
4	Frontend F0 and F1 (shared code, shell, session)	Me	Nothing, can start now
5	Frontend F2 and F3 (buyer, farmer, transporter routes)	Me	F1
6	Small backend additions (GET /me, GET /orders/{id}/payments)	Me	Nothing
7	Frontend F4 and F5	Me	Steps 2 and 6
8	Decide fee, transport price rule, silent buyer, disputes, then start Stage 2	You, then me	Steps 1 to 3